Assessment Submission

Candidate: Precious Enuagwune

Context Chosen: Retail and Order Enquiries

A. Problem and Solution Brief

The business problem: Zuri Home & Style is a fictional, growing online retailer of home goods and accessories in Lagos. Almost every customer request arrives on WhatsApp, and three staff (sales, dispatch and a manager) answer by hand. They handle about 150 enquiries a day: price and stock questions, order status, delivery questions, payment confirmations and complaints. Replies are slow, order details get lost between messages, follow-up is inconsistent, and the manager cannot see what has been handled, escalated or left unresolved. The business wants an affordable first stage that answers routine questions at once and sends everything else to the right person, without removing human judgement.

Users:

- Customers want a price, an order status or delivery information, or want to place an order or complain.
- Operations staff (sales and dispatch) receive escalated cases, verify payments and fulfil orders.
- The manager needs volume, escalation rate, response times and overdue cases at a glance.

Assumptions:

- About 150 inquiries a day and about 20 products. At this volume a single PostgreSQL instance is more than adequate.
- Customers pay by bank transfer, and staff verify each payment by hand.
- Delivery covers Lagos, Abuja and Port Harcourt at fixed fees by zone.
- Staff work 8am to 8pm WAT, Monday to Saturday. Automated replies work at any hour; overdue-case reminders are sent only during business hours.
- Customers are identified by phone number only. Order enquiries are answered only to the number that placed the order.
- All data is fictional or synthetic, and the WhatsApp channel is simulated with HTTP requests.

Constraints: A small technical team and low recurring cost. The first version covers a few high-value workflows only. Human escalation must stay available for sensitive or uncertain cases. Another engineer must be able to understand and maintain the system.

Scope: what the system does The system recognises six intents and answers or guides each one:

- **product_enquiry**: Looks up price and stock in the product table and replies.
- **order_status**: Validates the order ID (format ZH-12345), checks it belongs to the sender and replies with its status.

- **place_order**: Guided multi-step intake (product, quantity, delivery address, name, confirmation), then creates the order and alerts sales to confirm payment.
- **delivery_info**: Replies with zones, fees and timelines.
- **payment_info**: Replies with bank details. A message saying the customer has paid goes to staff.
- **complaint_refund**: Always escalated to the manager as high priority.

Unrecognised or low-confidence messages, requests for a person, and images or voice notes go to a human. Every message and decision is written to an audit log, and a dashboard shows volume by intent, escalation rate, unresolved cases and response times.

Intended outcomes:

- Routine questions are answered in seconds, at any hour.
- Every exception reaches the right staff member with a 30-minute response target, and overdue cases are chased automatically.
- The manager can see what is handled, escalated or unresolved without asking staff.
- Every message and decision can be traced in the audit log.

Deliberately not automated:

- Payment verification: staff check each transfer receipt.
- Complaints and refunds: always handled by a person.
- Stock reservation and fulfilment: the system records the order; staff confirm and dispatch.
- Courier (3PL) integration: delivery status comes from internal order statuses.
- Images and voice notes: forwarded to staff, not interpreted.
- Natural-language AI: classification uses deterministic keyword scoring, which is predictable, free to run and easy to audit.

B. System Architecture

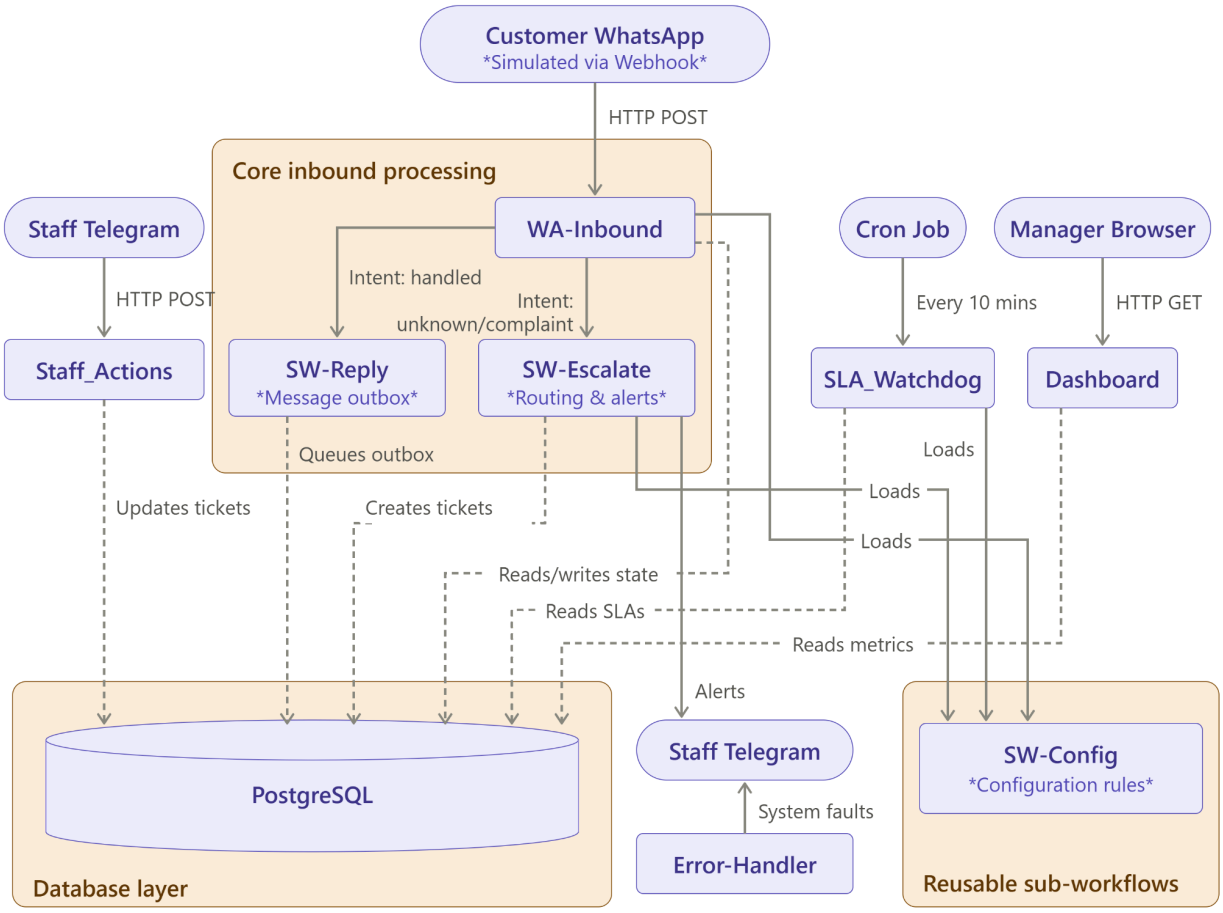


Figure 1. Dashed boxes are simulated in this build: the customer message is an HTTP POST in WhatsApp Cloud API format, and staff actions are HTTP requests. In production the message arrives from Meta's WhatsApp Cloud API and staff actions come from Telegram buttons. The dotted arrow is database access. Every workflow loads its rules from SW-Config, and a fault in any workflow goes to Error-Handler.

- **Triggers and logic:** WA-Inbound receives the webhook, validates and de-duplicates the message, classifies the intent and runs the handlers. SW-Config (rules and message wording), SW-Reply (outbox) and SW-Escalate (routing and alerts) are shared sub-workflows.
- **Data storage:** PostgreSQL, in its own Docker container and not exposed outside the Docker network, holds products, orders, contacts, messages, conversation_state, escalations, outbox and audit_log.
- **Notifications and human escalation:** SW-Escalate assigns the case by category (complaints to the manager, payments and orders to sales), sets a 30-minute response

deadline and alerts that person on Telegram. SLA-Watchdog runs every 10 minutes during business hours and reminds the manager about overdue cases. Staff claim and resolve cases through the Staff-Actions webhook.

- **Logging:** every step writes to audit_log with a request ID. The Dashboard reads it.
- **Failure handling:** duplicate message IDs are blocked by a UNIQUE database constraint and logged; low-confidence messages escalate to a human; a failed Telegram alert sets notify_status=failed, is logged and shows on the dashboard; any other runtime fault goes to Error-Handler, which alerts the administrator.
- **Working vs simulated:** fully working are all workflows, the database, classification, order intake, escalation routing, Telegram alerts (real test bot), the watchdog, the dashboard and the audit log. Simulated are the inbound WhatsApp message, the outgoing WhatsApp reply (written to the outbox table) and staff claim/resolve actions.
- **Why these tools:** n8n is visual and readable by a non-developer maintainer, with no licence fee. PostgreSQL is free, runs in the same Docker Compose, handles concurrent writes safely and is already what n8n itself uses, so there is one database technology to maintain. Keyword scoring is predictable, free per message and easy to audit. Telegram is free and instant.

E. Cost and Commercial Feasibility

- Setup (one-off): N300,000 suggested fee — About 5 days: configuration, loading the product catalogue, message testing, staff training and handover.
- Hosting: $4 to $10 a month (about N6,000 to N15,000) — One Server. A 2 GB plan is recommended for n8n.
- Domain and SSL: About $15 a year (about N1,900 a month) — Free Let's Encrypt certificate.
- Telegram alerts: N0 — The Bot API is free.
- WhatsApp messages: Variable — The main cost driver, and the only cost that grows with use. Infrastructure comes to roughly N8,000 to N17,000 a month, which is affordable for an SME of this size.

WhatsApp message fees:

Meta charges per delivered message, at a rate set by the customer's country code. Meta's pricing page lists free-form replies inside the 24-hour customer-service window as free. Reports from October 2026 say such replies may become chargeable at the utility rate, about $0.0101 (about N15) per message to Nigerian numbers. This is unconfirmed, so the design treats message fees as a pass-through cost. The estimate below uses 150 enquiries a day and about 3 bot replies per enquiry (about 13,500 replies a month):

- Replies free (current Meta page): WhatsApp fees per month: N0 | Cost per enquiry: N0
- Replies charged at about $0.0101: WhatsApp fees per month: About N150,000 | Cost per enquiry: About N45

If replies become chargeable, message fees dominate the bill. The main levers are shorter replies and fewer confirmation messages. Each customer message that opens or keeps the window open is free.

Pricing approach:

- One-off setup fee: N300,000 for build, configuration, testing, training and handover.
- Monthly managed service: N50,000 to N100,000, covering hosting, backups, monitoring, updates and small changes.
- WhatsApp message fees: passed through at cost, or an included allowance with overage. This keeps the client's bill predictable and protects the provider if Meta changes its rates.
- Cost per transaction: hosting and maintenance cost a few naira per enquiry at 150 enquiries a day. WhatsApp fees are the only variable part.

Sources: Meta, Pricing on the WhatsApp Business Platform (developers.facebook.com); trade press reports, October 2026 (Yournotify, MSME Africa Online). Verify current rates before contracting.

F. Risk, Reliability, and Security Note

1. **Personal data and the NDPA 2023**: The system handles phone numbers, names, delivery addresses and message text. The SME is the data controller and the builder a processor. Mitigation: operational tables hold only an HMAC-SHA256 hash of the phone number. One contacts table keeps the real number and rows are deleted after 90 days. Message text is deleted after 90 days, and n8n execution data is pruned after 7 days.
2. **Webhook spoofing**: In this simulated build the webhook accepts any POST. Mitigation: enforce HMAC-SHA256 validation of the X-Hub-Signature-256 header against the Meta app secret, complete Meta's verification handshake, and rate-limit at the reverse proxy.
3. **Wrong automatic answers and malformed input**: Keyword scoring can misclassify messages. Mitigation: confidence score combines keyword strength with the gap to the next-best intent; anything below 0.6 goes to a human. Complaints, refunds and payment confirmations always go to staff.

4. **Provider dependency**: Alerts depend on Telegram, and customers depend on WhatsApp and Meta. Mitigation: if Telegram is down, cases stay in the escalations table and remain visible on the dashboard.
5. **One server and one database (service continuity)**: A single VPS running n8n and PostgreSQL is a single point of failure, and the data lives in one Docker volume. Mitigation: a container restart policy, nightly pg_dump backups with an off-server copy, a documented restore, and a managed or replicated PostgreSQL as the production path.
6. **Auditability and staff misuse**: Staff could mark a case resolved without replying. Mitigation: audit log records who claimed or resolved each case and when.
7. **Access control and secrets**: Dashboard uses Basic Auth and Staff-Actions requires a header token. Secrets live in environment variables. PostgreSQL is not published outside the Docker network.

G. Operations Runbook

Start the system:

1. Install Docker and Docker Compose and clone the repository.
2. Copy .env.example to .env and set PHONE_PEPPER, N8N_ENCRYPTION_KEY, the database user, password and name (DB_POSTGRESDB_*), the timezone and the execution-pruning settings.
3. Run docker compose up -d. PostgreSQL is not published outside the Docker network.
4. Create the Zuri database once: docker compose exec postgres createdb -U <user> zuri.
5. Run PHONE_PEPPER=... ./db/setup.sh --reset. It creates the tables and loads synthetic seed data inside the container.
6. In n8n create four credentials: Telegram bot, Postgres (host is the compose service name, database zuri), Header Auth (Staff-Actions) and Basic Auth (Dashboard).
7. Import the workflow JSON files. Re-select the sub-workflow in every Execute Workflow node (IDs change on import), re-select the Postgres credential wherever n8n flags it, set each workflow's Error Workflow to Error-Handler, then activate them.

8. Open the Test-Runner workflow in n8n and click Execute workflow, or optionally run run-tests.sh in tests folder (for terminal run only).

Monitor:

- Business view: open the Dashboard. Check unresolved cases, SLA breaches and the failure panel.
- System health: Error-Handler sends workflow name, failed node, error message and execution link to Telegram. Scan n8n Executions list daily.
- Trace any request: SELECT ts, event, detail FROM audit_log WHERE request_id='...'

H. Handover and 90-Day Plan

Handover to a Junior Engineer

- **Day 1:** read the README and Section B, then run the setup and the test script on their own machine with the synthetic data.
- **SW-Config first:** all routing rules, hours, fees, keywords and message wording live there, so a junior can change the bot's behaviour without touching workflow logic.
- **Trace a message:** follow one message through the n8n Executions view from WA-Inbound to the reply, then one escalation through to the Telegram alert and Staff-Actions.
- **First tasks (low risk):** edit a reply message, add a delivery zone, add a keyword, add a test case.
- **Safe review:** the junior changes a copy of a workflow, exports the JSON to a git branch and runs ./tests/run-tests.sh. I review the diff and the test output before it replaces the live workflow. Nobody edits live workflows directly. Changes to SW-Escalate, SW-Reply, credentials or the database schema are reviewed with me present.
- **First-line support:** the junior works from the runbook and escalates database and credential problems to me.

Improvement Plan

- **First 30 days (production hardening)**: move from the simulated webhook to Meta's WhatsApp Cloud API: verification handshake, HMAC signature check, test number and approved message templates. Replace inlined SQL with parameterised queries and add a database trigger so audit_log is append-only. Automate backups and the retention purge as a scheduled workflow, and add external uptime monitoring. Merge repeat messages into the open escalation instead of creating a new one.
- **Days 31 to 60 (staff experience)**: store image IDs with each case so staff see payment proof. Replace the curl-based staff endpoint with Telegram claim and resolve buttons. Put the dashboard behind HTTPS with per-user logins. Add approved templates for

follow-ups outside the 24-hour window, and review reminder rules with the manager using real response-time data.

- **Days 61 to 90 (scale and automation)**: review escalation rate and misclassified messages, and tune the keywords. Review database size and query times, and move PostgreSQL to a managed or replicated service if the client needs higher availability. Integrate a local payment gateway such as Paystack to automate payment verification, keeping human review for mismatches. Compare actual WhatsApp fees against the cost model.